DevOps — Zero to Master

23 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 What is DevOps?

02 Linux Basics

03 Networking Fundamentals

04 Shell Scripting

05 Git and Version Control

06 Web Servers

Intermediate (6 topics)

07 CI/CD Concepts and Tools

08 Docker Fundamentals

09 Dockerfile Best Practices

10 Docker Compose

11 Cloud Basics (AWS/Azure/GCP)

12 Package and Artifact Management

Advanced (7 topics)

13 Kubernetes Fundamentals

14 Helm and Kustomize

15 Infrastructure as Code: Terraform

16 Configuration Management: Ansible

17 Monitoring and Logging

18 Security and DevSecOps

19 GitOps

Master (4 topics)

20	SRE: SLIs, SLOs and Incident Response
21	Release Strategies
22	Scalability, Cost and Reliability
23	Real Project: End-to-end Pipeline

Beginner Level

01. What is DevOps?

DevOps unites development and operations to ship software faster and more reliably through automation, collaboration and continuous feedback (CI/CD, IaC, monitoring).

02. Linux Basics

Navigate and manage systems from the command line: files, permissions, processes, packages, services.

```
ls -la
chmod 644 file
ps aux | grep nginx
systemctl status nginx
```

03. Networking Fundamentals

IP, DNS, ports, HTTP/HTTPS, TCP vs UDP, load balancers, firewalls, SSH.

```
curl -I https://example.com
ss -tulpn
```

04. Shell Scripting

Automate tasks with Bash: variables, loops, conditions, functions, exit codes.

```
#!/usr/bin/env bash
set -euo pipefail
for f in *.log; do gzip "$f"; done
```

05. Git and Version Control

Branching, pull requests and trunk-based development are the base of automation.

```
git switch -c feature/x
```

06. Web Servers

Nginx/Apache as web servers and reverse proxies; TLS certificates with Let's Encrypt.

```
server { listen 80; location / { proxy_pass http://127.0.0.1:3000; } }
```

Intermediate Level

07. CI/CD Concepts and Tools

Continuous Integration builds and tests every change; Continuous Delivery/Deployment releases automatically. Tools: GitHub Actions, GitLab CI, Jenkins.

```
on: push
jobs:
  build:
    runs-on: ubuntu-latest
```

08. Docker Fundamentals

Containers package apps and dependencies. Images, containers, volumes, networks, Dockerfile.

```
docker build -t myapp .
docker run -d -p 8080:80 myapp
```

09. Dockerfile Best Practices

Small base images, multi-stage builds, layer caching, non-root users, `.dockerignore`.

```
FROM node:22-alpine AS build
WORKDIR /app
COPY . .
RUN npm ci && npm run build
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
```

10. Docker Compose

Define multi-container apps (app + database + cache) in YAML.

```
services:
  web:
    build: .
    ports: ["3000:3000"]
  db:
    image: postgres:16
```

11. Cloud Basics (AWS/Azure/GCP)

Compute, storage, networking, IAM, managed databases; shared responsibility model.

```
aws s3 ls
```

12. Package and Artifact Management

Registries for images and packages; versioning, tagging and immutability.

```
docker tag myapp:1.0 registry.example.com/myapp:1.0
```

Advanced Level

13. Kubernetes Fundamentals

Pods, Deployments, Services, ConfigMaps, Secrets, Ingress; declarative manifests managed with `kubectl`.

```
kubectl apply -f deployment.yaml
kubectl get pods
```

14. Helm and Kustomize

Package and customise Kubernetes manifests per environment.

```
helm install myapp ./chart -f values-prod.yaml
```

15. Infrastructure as Code: Terraform

Provision cloud infrastructure with declarative code; state, modules, plan/apply.

```
terraform init
terraform plan
terraform apply
```

16. Configuration Management: Ansible

Agentless automation with playbooks and roles.

```
- hosts: web
  tasks:
    - apt: name=nginx state=present
```

17. Monitoring and Logging

Metrics with Prometheus/Grafana, logs with ELK/Loki, alerts and dashboards; the golden signals.

```
rate(http_requests_total[5m])
```

18. Security and DevSecOps

Shift-left security: SAST/DAST, dependency and image scanning, secrets management (Vault), least privilege.

```
trivy image myapp:1.0
```

19. GitOps

Git as the single source of truth with Argo CD or Flux reconciling clusters.

```
argocd app sync myapp
```

Master Level

20. SRE: SLIs, SLOs and Incident Response

Define reliability targets, error budgets, on-call, runbooks and blameless postmortems.

21. Release Strategies

Blue-green, canary and rolling deployments, feature flags, rollbacks.

22. Scalability, Cost and Reliability

Autoscaling, caching, CDN, multi-region, backups and disaster recovery, FinOps.

23. Real Project: End-to-end Pipeline

Code -> CI tests -> Docker image -> push to registry -> deploy to Kubernetes via Terraform + Argo CD, with monitoring and alerts.

Interview Question Bank

Q1. What is DevOps?
A culture and set of practices combining development and operations to deliver software quickly and reliably.

Q2. CI vs CD?
CI merges and tests code frequently; CD automatically delivers (or deploys) validated builds.

Q3. Docker vs virtual machine?
Containers share the host kernel and are lightweight; VMs include a full guest OS.

Q4. What is Infrastructure as Code?
Managing infrastructure through versioned, declarative code (Terraform, CloudFormation).

Q5. What is Kubernetes?
A container orchestrator handling scheduling, scaling, networking and self-healing.

Q6. Blue-green vs canary deployment?
Blue-green switches traffic between two environments; canary releases to a small percentage first.

Q7. What is a reverse proxy?
A server that forwards client requests to backend servers (Nginx), adding TLS, caching and load balancing.

Q8. What is GitOps?
Declaring desired state in Git and having an operator reconcile the cluster to it.

Q9. What are the golden signals?
Latency, traffic, errors, saturation.

Q10. How do you manage secrets?
Secret managers (Vault, cloud KMS), environment injection, never in Git.

Capstone Project Ideas

1. Dockerise a full-stack app
2. CI/CD with GitHub Actions to a VPS
3. Terraform an AWS VPC + EC2
4. Kubernetes deployment with Helm
5. Prometheus + Grafana monitoring